2º DAW - Optativa “J2EE y Spring Boot”

Unidad 1 - HTTP y J2EE

Bloque 3 - Nuevo proyecto maven + web en eclipse + tomcat v11

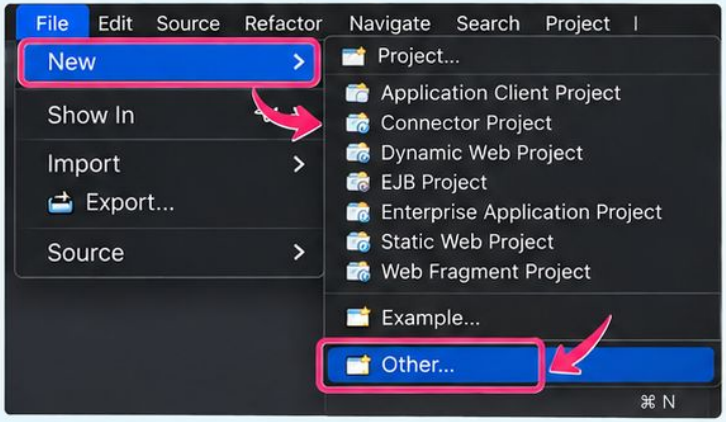
CREAR UN PROYECTO MAVEN
en Eclipse (paso 1 de 6)

El primer paso para nuestra aplicación web

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

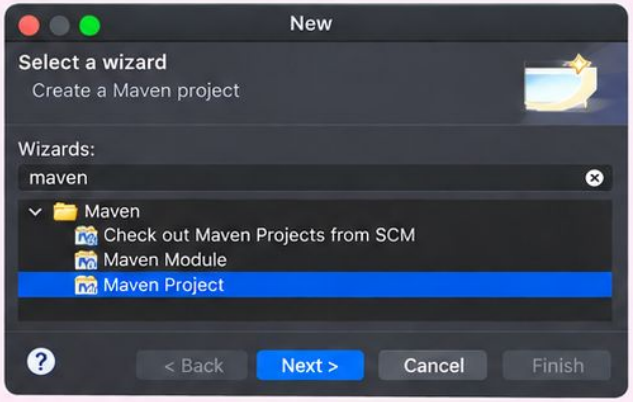
1 Abrir el asistente de nuevo proyecto

En Eclipse, selecciona **File → New → Other...**



2 Seleccionar un proyecto Maven

En el asistente, despliega la carpeta **Maven** y selecciona **Maven Project.**



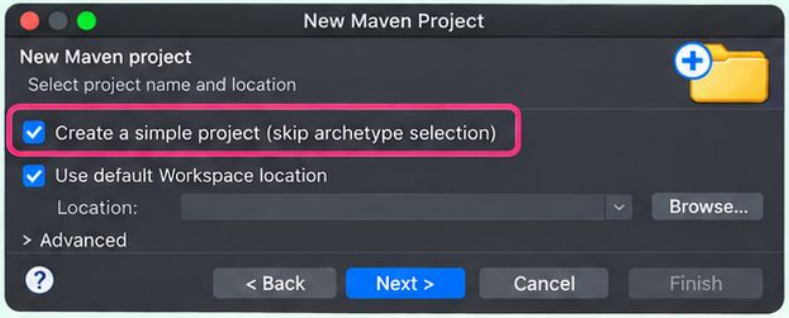
TEN EN CUENTA

Vamos a crear primero un proyecto Maven. En los siguientes pasos lo convertiremos en un proyecto web para poder desplegarlo en Tomcat 11.



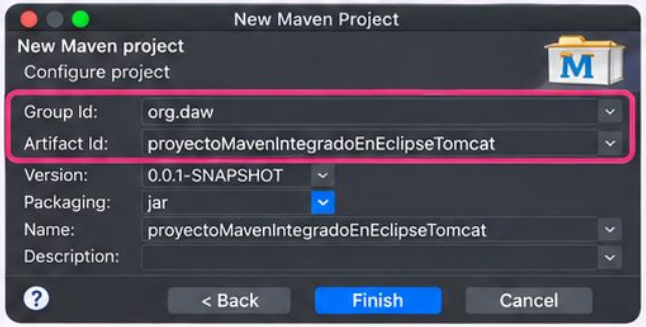
3 Crear un proyecto sencillo (sin arquetipo)

Marca la opción **Create a simple project (skip archetype selection)** y pulsa **Next >**.



4 Especificar el Group Id y el Artifact Id

Introduce los datos de identificación del proyecto.
Puedes dejar el resto de opciones con los valores por defecto.

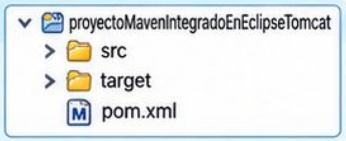


5 Crear el proyecto

Pulsa **Finish** para crear el proyecto Maven. Eclipse generará la estructura de carpetas y el fichero pom.xml.

Resultado

En el explorador de proyectos verás un proyecto Maven con su estructura básica y el fichero pom.xml.



Siguiente paso

En la siguiente infografía convertiremos este proyecto en un proyecto web (Dynamic Web Module).

CONVERTIR MAVEN EN PROYECTO WEB
en Eclipse (paso 2 de 6)

Añadimos la faceta web para poder desplegarlo en Tomcat 11

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

TEN EN CUENTA

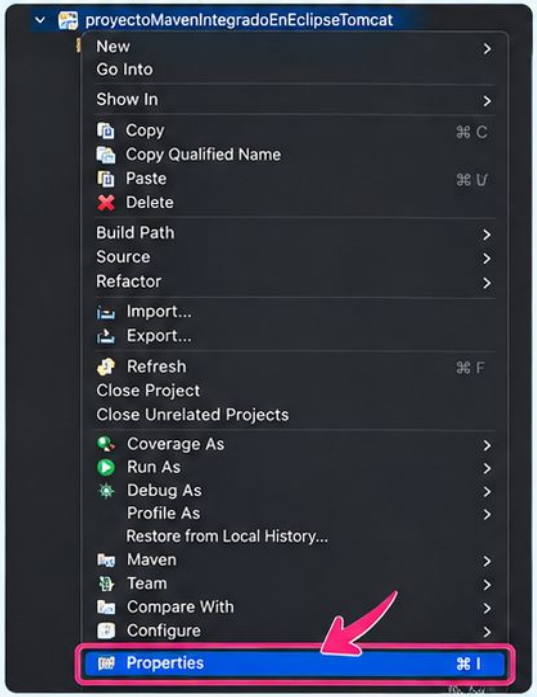
Un proyecto Maven por sí solo no es un proyecto web.

Debemos añadir la faceta "Dynamic Web Module" para que Eclipse lo trate como una aplicación web y pueda desplegarse en Tomcat 11.



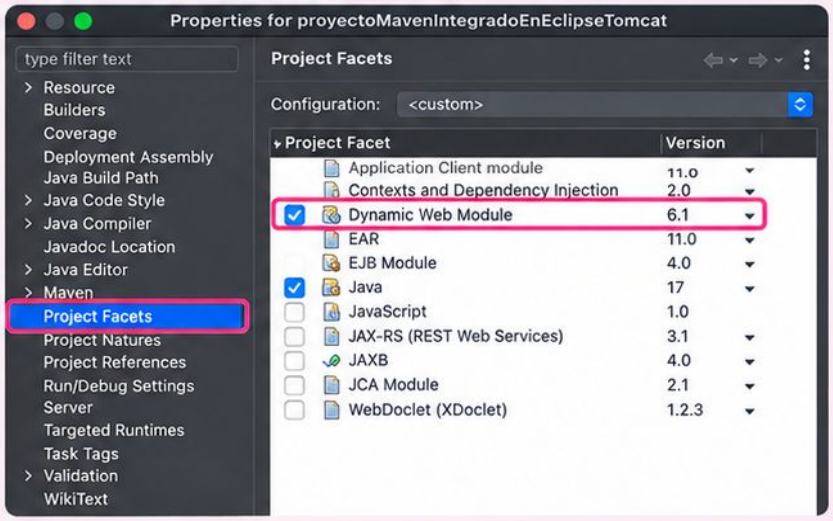
1 Abrir las propiedades del proyecto

En el Explorador de proyectos, haz clic derecho sobre el proyecto y selecciona **Properties.**



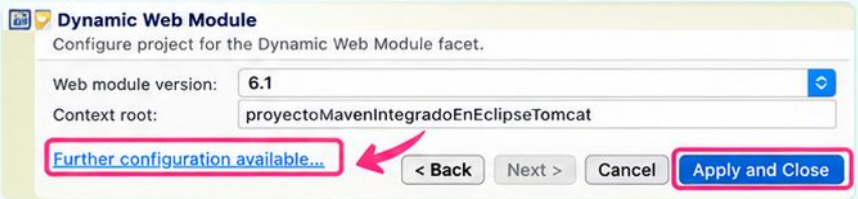
2 Seleccionar las facetas del proyecto

En la ventana de propiedades, selecciona **Project Facets** y marca las facetas **Java** y **Dynamic Web Module** (versión 6.1).



3 Configurar la carpeta de contenido web

En la faceta "Dynamic Web Module" pulsa el enlace **Further configuration available...**



4 Especificar la carpeta webapp

En la configuración adicional, asegúrate de que la carpeta de contenido es **src/main/webapp.** Después pulsa **Apply and Close.**

5 Aplicar los cambios

Pulsa **Apply and Close** en la ventana de propeidades. A partir de este momento, Eclipse tratará el proyecto como una aplicación web.

Resultado

El proyecto ya es un proyecto web Maven.
La carpeta src/main/webapp será la raíz de los recursos estáticos (HTML, CSS, JavaScript, ...).



Siguiente paso

En la siguiente infografía crearemos un fichero index.html en la carpeta webapp para comprobar que todo funciona.

ESTRUCTURA DE UN PROYECTO WEB MAVEN
en Eclipse (paso 3 de 6)

Así se organiza nuestro proyecto

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

TEN EN CUENTA

Un proyecto web Maven tiene una estructura de carpetas especifica.

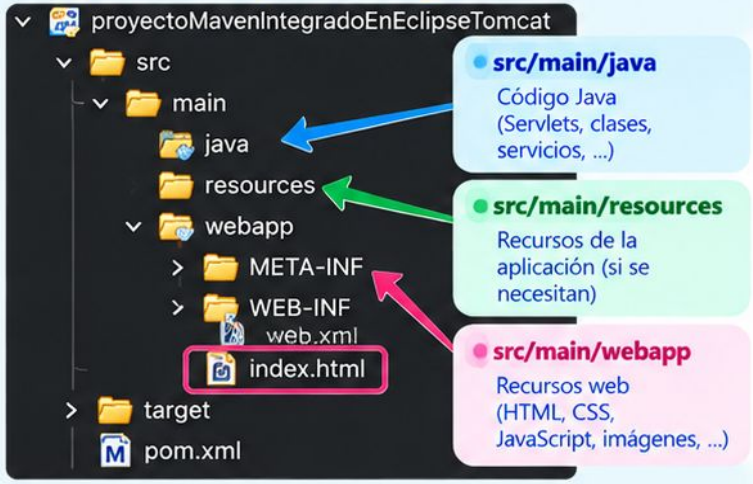
Cada tipo de archivo tiene su lugar:

- El código Java en **src/main/java**.
- Los recursos web en **src/main/webapp**.
- La configuración del proyecto en el **pom.xml**.



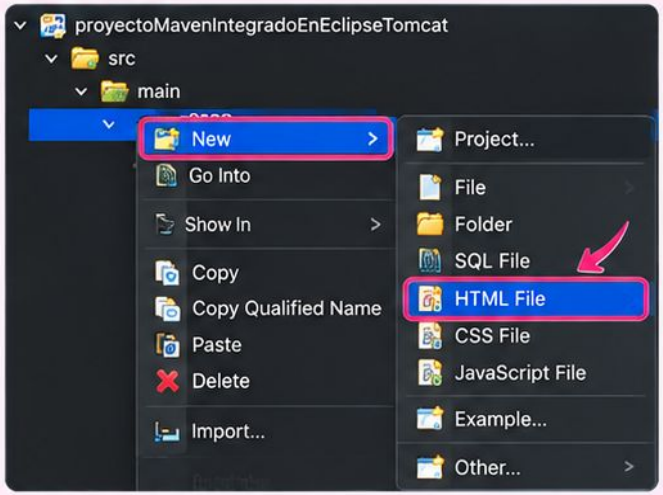
1 Estructura del proyecto

Tras convertir el proyecto en una aplicación web, la estructura de carpetas debe ser parecida a la siguiente.



2 Crear un fichero index.html

En la carpeta **src/main/webapp** podemos crear un nuevo fichero HTML.



3 Contenido del index.html

Cualquier fichero **index.html** sevirá. Por ejemplo:

index.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5  </head>
5  <body>
6      Hola mundo
8  </body>
7  </html>
```

4 Estructura y función de cada carpeta

src/main/java — Código Java de la aplicación:
- Servlets
- Clases
- Servicios
- ...

src/main/resources — Recursos de la aplicación (si se necesitan).

src/main/webapp — Archivos estáticos de la aplicación web:
- HTML
- CSS
- JavaScript
- Imágenes
- ...

pom.xml — Configuración del proyecto y dependencias Maven.

5 Comprobar en el navegador

Iniciamos Tomcat 11 y accedemos al fichero **index.html** desde un navegador web.



Resultado

Hemos creado un fichero **index.html** en **src/main/webapp** y Tomcat lo sirve correctamente.

INTEGRAR EL PROYECTO CON TOMCAT 11
en Eclipse (paso 4 de 6)

El proyecto no se sirve automáticamente con Tomcat

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

TEN EN CUENTA

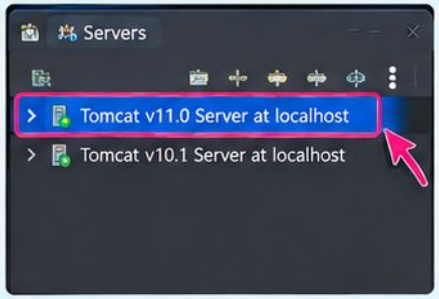
Crear el proyecto no significa que Tomcat lo esté sirviendo todavía.

Debemos añadir el proyecto a la configuración del servidor Tomcat 11 y arrancarlo.



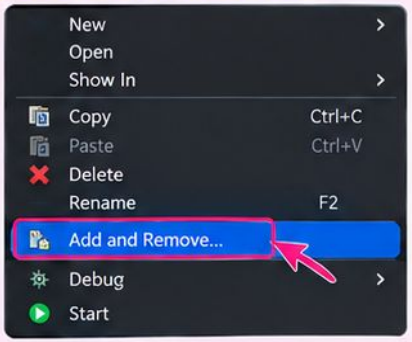
1 Abrir la vista Servers

En la vista Servers, localiza el servidor **Tomcat v11.0 Server at localhost**.



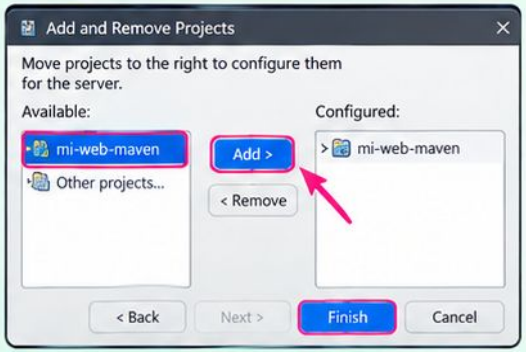
2 Menú contextual

Haz clic con el botón derecho sobre el servidor y selecciona **Add and Remove...**.



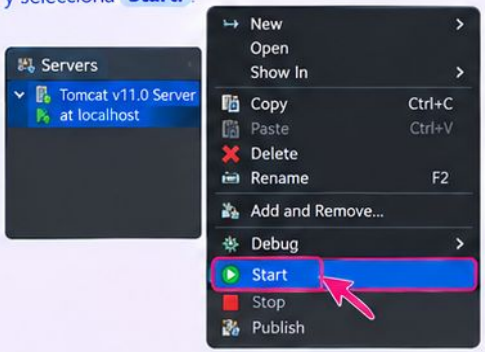
3 Añadir el proyecto

En la ventana "Add and Remove Projects" mueve tu proyecto Maven de **Available** a **Configured**.



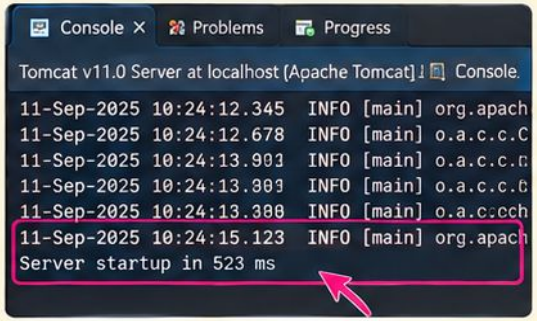
4 Arrancar Tomcat

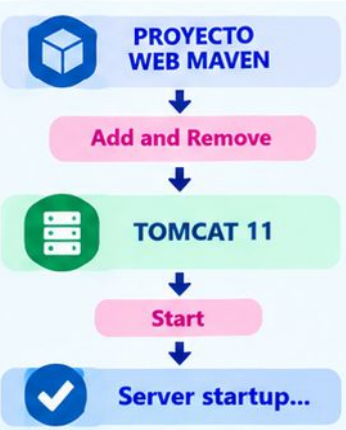
En la vista Servers, haz clic derecho sobre el servidor y selecciona **Start**.



5 Comprobar la consola

En la consola de Tomcat debes ver un mensaje como este:





Resultado

El proyecto queda desplegaldo en Tomcat 11 y el servidor arranca correctamente, sin errores.

Si Tomcat no arranca correctamente, no continúes con las pruebas del Servlet.

Recuerda

En la siguiente unidad ya podremos comprobar que el proyecto se sirve correctamente desde el navegador.

AÑADIR JAKARTA SERVLET AL PROYECTO
en Eclipse (paso 5 de 6)

La API de Servlet para Tomcat 11

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

TEN EN CUENTA

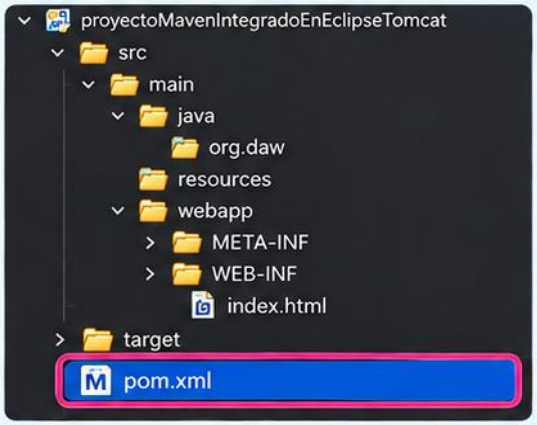
Para desarrollar Servlets necesitamos la API de Jakarta Servlet.

Tomcat 11 utiliza Jakarta **EE**, por lo que debemos incluir la dependencia **jakarta.servlet-api** en nuestro **pom.xml**.



1 Abrir el fichero pom.xml

En el Explorador de proyectos, abre el fichero **pom.xml** de nuestro proyecto.



2 Añadir la dependencia

En el fichero **pom.xml**, dentro de la sección **<dependencies>**, añadimos la dependencia de Jakarta Servlet con los siguientes datos.

pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
  ...
  <dependencies>
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.1.0</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  ...
</project>
```

3 ¿Qué significa cada elemento?

jakarta.servlet	Grupo de la API de Servlet de Jakarta EE (para Tomcat 11).
jakarta.servlet-api	Artefacto que contiene las interfaces de Servlet.
6.1.0	Versión de la API, compatible con Tomcat 11.
<scope>provided</scope>	La aplicación necesita esta API para compilar, pero Tomcat proporciona su implementación en tiempo de ejecución.

4 Tomcat 11 y Jakarta Servlet



Tomcat 11
Utiliza Jakarta EE

↓

Jakarta Servlet
API de servlets y filtros para aplicaciones web

↓

Paquetes que usaremos
`jakarta.servlet.*`

5 No usar javax.servlet

Incorrecto en Tomcat 11

```
import javax.servlet.*;
```

javax.servlet se utiliza en versiones antiguas de Java EE y no es compatible con Tomcat 11.

Correcto en Tomcat 11

```
import jakarta.servlet.*;
```

Debemos usar **jakarta.servlet**, que es la versión actual en Jakarta EE y compatible con Tomcat 11.

Resultado

El proyecto ya tiene incluida la dependencia de **jakarta.servlet-api** en el pom.xml y podemos utilizar las clases **jakarta.servlet.*** en nuestros Servlets.

Fragmento completo

```xml
<dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
</dependency>
```

Siguiente paso

En la siguiente infografía crearemos nuestro primer Servlet usando la API de Jakarta Servlet.

CREAR, CONFIGURAR Y PROBAR EL PRIMER SERVLET

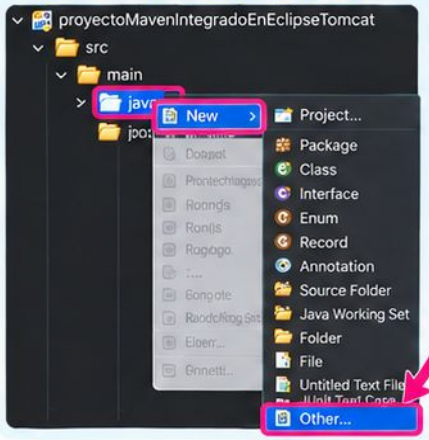
en Eclipse (paso 6 de 6)

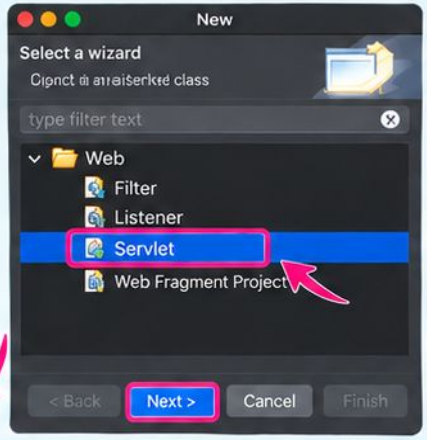
¡Ya tenemos nuestra primera petición HTTP atendida por un Servlet!

En esta unidad...
- Proyecto Maven
- Proyecto Web
- Tomcat 11

1 Crear el Servlet

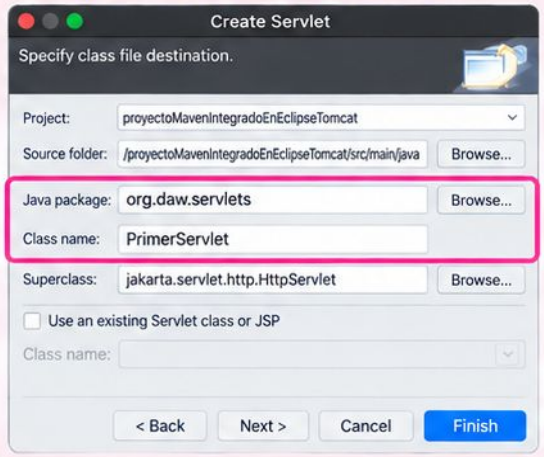
Desde la carpeta **src/main/java**, creamos un nuevo **Servlet** usando el asistente de Eclipse.





2 Configurar el Servlet

Completamos los datos del asistente.



¡IMPORTANTE!

La URL del Servlet y el nombre de la clase **NO** tienen por qué ser iguales.



3 Revisar los imports y el código generado

El asistente puede generar imports de **javax.servlet**. Debemos sustituirlos por **jakarta.servlet**, compatible con Tomcat 11.

Incorrecto (javax.servlet)

```
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http:HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

Correcto (jakarta.servlet)

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http:HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

Código del Servlet (ejemplo)

```
package org.daw.servlets;

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/miServlet")
public class PrimerServlet extends HttpServlet {

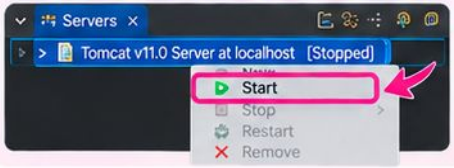
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<h1>Hola desde
        miServlet</h1>");
    }
}
```

La URL del Servlet puede tener un nombre diferente al de la clase.

4 Ejecutar y probar el Servlet

1. Iniciar Tomcat 11

Iniciamos el servidor desde la vista Servers.



2. Probar en la navegador

Accedemos a la URL del Servlet.



Resultado

El Servlet se ejecuta correctamente y la respuesta se muestra en el navegador.

Flujo de la petición

